

Doubly Linked List – Delete From End – Space Complexity and Algorithm.

Program :

```
void deleteFromEnd()
{
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }
    Node *temp = head;
    if (temp->next == nullptr)
    { // Only one node in the list
        head = nullptr;
    }
    else
    {
        while (temp->next != nullptr)
        {
            temp = temp->next; // Traverse to the last node
        }
        temp->prev->next = nullptr;
        temp->prev = nullptr;

        cout << "Deleted " << temp->data << " from the end." <<
endl;
        free(temp);
        temp = nullptr;
    }
    ... .
case 6:
    deleteFromEnd();
    break;
```

Space Complexity

Auxiliary Space

Auxiliary space is the extra memory used by a function, excluding the space taken by the inputs themselves.

In the deleteFromEnd function, the only new variable created is:

*Node *temp: A single temporary pointer used to traverse the list and hold the address of the node being deleted. The size of this pointer is constant.*

The function's main memory operation is free(temp), which deallocates memory. The while loop affects the function's time complexity but not its space complexity. Since the extra memory used does not grow with the list's size, the auxiliary space is $O(1)$.

Auxiliary Space: $O(1)$

Input Space

Input space is the memory used to store the function's inputs. This can be analyzed from two perspectives.

1. Parameter – only Definition

This view considers only the space taken by the function's direct parameters. The function signature is void deleteFromEnd().

The function takes no parameters.

Under this definition, the input space is constant.

Input Space (Parameter-only): $O(1)$

2. Full – Data Definition

This broader view includes the space taken by the entire data structure the function operates on—the doubly linked list.

If the linked list contains n nodes before the deletion, the total memory it occupies is proportional to n .

Under this definition, the input space is linear with respect to the size of the list.

Input Space (Full-Data): $O(n)$

Total Space Complexity

The total space complexity is the sum of the auxiliary space and the input space. The result depends on which definition of input space you use.

- **Common Interpretation (using parameter-only input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In most academic and interview contexts, the "space complexity" of a function refers to its auxiliary space. Therefore, the most common and expected answer for this function is $O(1)$.

Algorithm: Delete from End (Doubly Linked List)

DDELEND(START)

1. **[CHECK IF LIST IS EMPTY]**

If START = NULL, then:

Write: "List is empty. Nothing to delete." and EXIT.

[End of If structure.]

2. **[INITIALIZE POINTER]**

Set PTR := START [PTR is a pointer that will find the last node].

3. **[CHECK FOR SINGLE NODE vs MULTI – NODE LIST]**

If NEXTLINK[PTR] = NULL, then: [List has only one node]

Set START := NULL [The list becomes empty].

ELSE: [List has more than one node]

a. Repeat while NEXTLINK[PTR] ≠ NULL: [Traverses the list to find the last node].

Set PTR := NEXTLINK[PTR] [Updates PTR to the next node].

[END of Loop]

b. Set NEXTLINK[PREVLINK[PTR]] := NULL (The second – to – last node's NEXT pointer is set to NULL, making it the new last node).

c. Set PREVLINK[PTR] := NULL [UNLINK PREV pointer of LAST NODE from 2nd LAST NODE]

4. **[DEALLOCATE THE ORIGINAL LAST NODE]**

Write: "Deleted", INFO[PTR], " from the end."

free(PTR) [Releases the memory of the node pointed to by PTR].

5. **[AVOID DANGLING POINTER]**

SET PTR := NULL.

6. **Exit**
-